

Software Engineering – UE24CS341A

Lab 3: Component Modelling & Architectural Pattern Selection

Name : Anand

SRN : PES1UG24AM341

Section : F

Project: OpenAPI Specification & Mock Generation :

1. Objective :

To evaluate different architectural styles, select the most appropriate architecture for the **OpenAPI Mock Server Generator**, and design a UML Component Diagram showing the major components, interfaces, dependencies, and interactions between them.

2. Scenario Overview :

The **OpenAPI Mock Server Generator** is a developer productivity tool that accepts **OpenAPI 3.0 YAML/JSON specifications** and generates dynamic HTTP mock REST endpoints.

The system allows API Developers to import and parse OpenAPI specifications, generate mock endpoints, validate incoming API requests against defined schemas, generate schema-compliant mock responses, configure simulated response latency, and view the generated endpoints.

3. Key Requirements :

Functional Requirements :

- API Developers can import OpenAPI 3.0 YAML/JSON specifications.
- The system parses OpenAPI paths, operations, parameters, and schema definitions.

- The system generates dynamic mock HTTP endpoints from valid specifications.
- Incoming API requests are validated against the corresponding request schemas.
- The system generates randomized, schema-compliant mock responses.
- API Developers can configure simulated response latency.
- API Developers can view the generated mock endpoints, HTTP methods, and paths.
- QA Engineers/API Clients can send requests to generated mock endpoints and receive mock responses.

Non – Functional Requirements :

- The mock server engine must sustain 1,000 mock API requests per second while maintaining configured latency within ± 10 ms.
- Generated mock endpoints should remain available throughout a testing session unless explicitly stopped or removed.

4. Architectural Style Analysis :

Architecture	Advantages	Disadvantages	Suitability
Layered Architecture	Clear separation of concerns, easy to understand and maintain	Some performance overhead and coupling between layers	High
Microservices Architecture	Independent scaling, fault isolation, independent deployment	Higher operational complexity, network latency, data consistency challenges	Medium
Client-Server Architecture	Centralized control, simple deployment and data consistency	Single point of failure and scalability bottleneck	Medium

5. Architecture Selection

Selected Architecture: Layered Architecture

We chose Layered Architecture for the OpenAPI Mock Server Generator because the system has clear separation between the presentation interface, OpenAPI processing and mock-server business logic, validation/response generation, and configuration/data management.

This architecture provides a simple and maintainable structure for implementing OpenAPI parsing, mock endpoint generation, request validation, response generation, endpoint configuration, and API testing.

Reasons for Selection :

Reason 1 — Separation of Responsibilities

The system contains different responsibilities such as OpenAPI specification parsing, mock endpoint generation, request validation, response generation, endpoint configuration, and endpoint viewing.

Layered Architecture separates these responsibilities into logical components, making the system easier to understand, test, maintain, and modify.

Reason 2 — Suitable for the Current System

The OpenAPI Mock Server Generator follows a structured application model consisting of a user interface, application/business processing, mock API processing, and configuration/data management.

Layered Architecture fits this structure without introducing the additional operational complexity associated with independently deployed microservices.

It also provides a clear flow from OpenAPI import → parsing → endpoint generation → request validation → mock response generation.

6. Security Advantage :

The validation component provides a security boundary between incoming API requests and the internal mock-processing components.

Incoming requests can be validated against the OpenAPI-defined request schemas before being processed by the mock endpoint engine. Invalid requests can therefore be rejected without being passed further into the system.

Separating the interface, processing, validation, and configuration components also helps restrict direct access to internal system functionality.

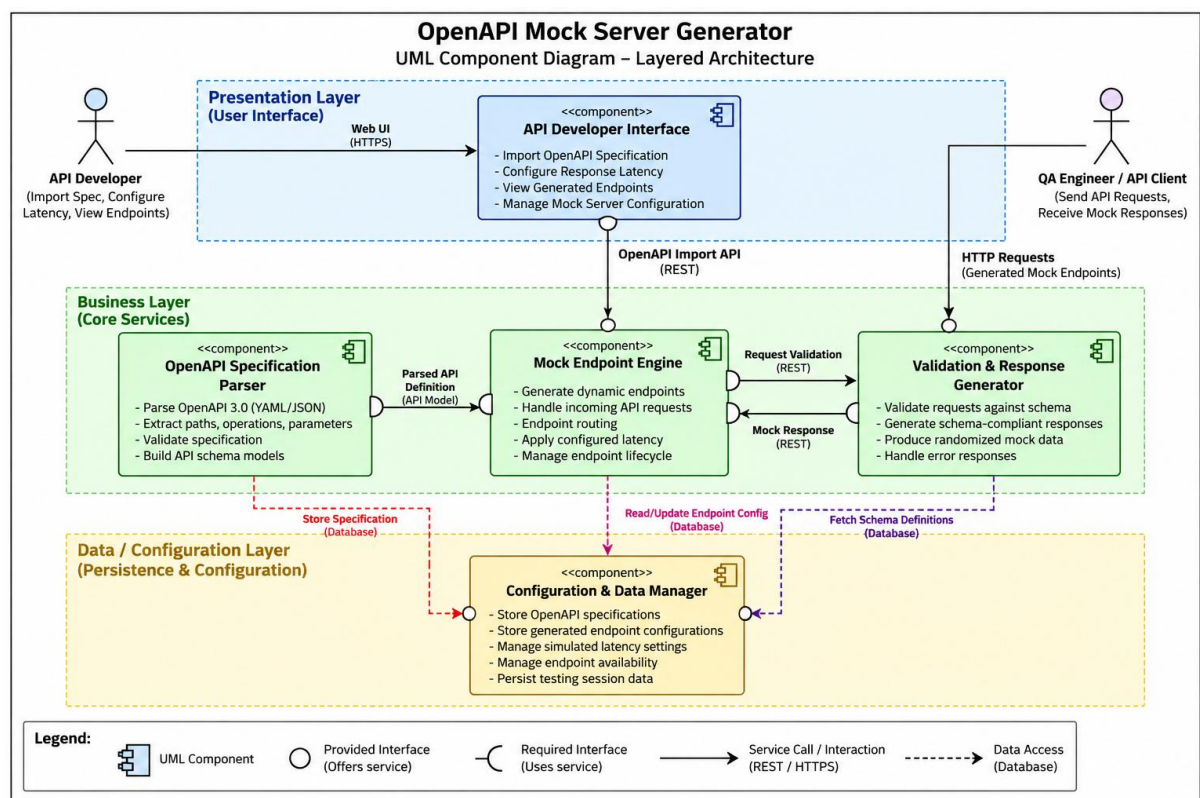
7. Performance Benefit

The layered structure allows request processing and mock response generation to be handled independently from the user interface.

The **Mock Endpoint Engine** can process incoming API requests directly through the validation and response-generation components without unnecessary interaction with the presentation layer.

This supports the system's performance requirement of handling up to **1,000 mock API requests per second** while maintaining configured simulated latency within **±10 ms**.

8. UML Component Diagram :



9. Component Interfaces and Interactions :

Interface	Components	Purpose
OpenAPI Import API	API Developer Interface ↔ OpenAPI Specification Parser	Imports OpenAPI 3.0 YAML/JSON specifications for parsing
Endpoint Generation Interface	OpenAPI Specification Parser ↔ Mock Endpoint Engine	Transfers parsed API paths, operations and schemas for endpoint generation
Request Validation Interface	Mock Endpoint Engine ↔ Validation & Response Generator	Sends incoming API requests for schema validation
Mock Response Interface	Validation & Response Generator ↔ Mock Endpoint Engine	Returns randomized schema-compliant mock responses
Configuration / Latency Interface	API Developer Interface ↔ Configuration & Data Manager	Stores and retrieves endpoint and simulated latency configuration
Endpoint Configuration Interface	Configuration & Data Manager ↔ Mock Endpoint Engine	Provides endpoint configuration, availability and latency information

10. Data Flow :

OpenAPI Import Flow :

API Developer

↓

API Developer Interface

↓

OpenAPI Specification Parser

↓

Parsed OpenAPI Specification

↓

Mock Endpoint Engine

↓

Generated Mock Endpoints

API Request Flow :

QA Engineer / API Client

↓

Generated Mock Endpoint

↓

Mock Endpoint Engine

↓

Validation & Response Generator

↓

Schema-Compliant Mock Response

↓

QA Engineer / API Client

Configuration Flow :

API Developer

↓

API Developer Interface

↓

Configuration & Data Manager

↓

Endpoint / Latency Configuration

↓

Mock Endpoint Engine

11. Conclusion :

Layered Architecture is suitable for the OpenAPI Mock Server Generator because it provides clear separation between the presentation interface, OpenAPI processing, mock endpoint generation, request validation, response generation, and configuration management.

The architecture supports the major functionality established in Labs 1 and 2, including OpenAPI specification parsing, mock endpoint generation, API request validation, schema-compliant responses, configurable response latency, endpoint visibility, high-volume request handling, and endpoint availability.